SISTEMAS DISTRIBUIDOS

PRÁCTICA 1 – Modelos de Comunicación y Middleware

PAU VALLS TARRATS Y LUIS MANUEL OCAMPO ÁVILA

Índice

Estructura del Documento 2
Estructura de la Práctica 2
Observaciones 2
Comunicación Directa 3
Entradas No Numeradas 3
Arquitectura 3
Implementación 4
Resultados 4
Entradas Numeradas 5
Arquitectura 5
Implementación 5
Resultados 5
Comunicación Indirecta 6
Entradas No Numeradas 6
Arquitectura 6
Implementación 7
Resultados 7
Entradas Numeradas 9
Arquitectura 9
Implementación 9
Resultados 10

Estructura del Documento

Este documento se divide en dos grandes grupos: *Comunicación Directa* y *Comunicación Indirecta*. Dentro de cada uno de ellos se encuentra nuestra propuesta detallada para solucionar el problema planteado. Además, cada grupo tiene dos apartados; *Entradas No Numeradas* y *Entradas Numeradas*, y en ellos se explican las estrategias seguidas según el sistema de venta de entradas.

Estructura de la Práctica

En esta sección se detalla brevemente cómo hemos organizado los ficheros de la práctica. Estos se encuentran en los directorios:

- `cd_un`: Contiene los archivos relativos a *Comunicación Directa* y *Entradas No Numeradas*.
- `cd_num`: Contiene los archivos relativos a *Comunicación Directa* y *Entradas Numeradas*.
- `ci_un`: Contiene los archivos relativos a Comunicación Indirecta y *Entradas No Numeradas*.
- `ci_num`: Contiene los archivos relativos a *Comunicación Indirecta* y *Entradas Numeradas*.
- `data`: Contiene los archivos de texto con las peticiones.

Observaciones

Para hacer las pruebas de ejecución de peticiones, el escenario consta de los siguientes elementos:

- Una RaspberryPI: Ubicación del Cliente.
- Un PC de sobremesa: En el caso de comunicación directa, ha contenido el servicio de nombres y el balanceador de carga.
- Un portátil: Ubicación de los trabajadores, y servicios como `rabbitmq` y `REDIS`.

Todos los elementos están conectados a través de una red LAN, y físicamente conectados al mismo conmutador.

Comunicación Directa

Entradas No Numeradas

Arquitectura

En este tipo de comunicación se requería un balanceador de carga. Hemos decidido ceñirnos a la arquitectura más sencilla que involucra un balanceador de carga. La arquitectura consta de los siguientes actores:

- **Cliente**: Solicita la compra de entradas.
- **Balanceador de carga**: Recibe las solicitudes y las distribuye entre los trabajadores asignados. Al recibir los resultados de los trabajadores, estos se envían de nuevo al cliente.
- **Trabajadores**: Reciben peticiones e incrementan el contador atómico. El resultado de la compra es satisfactorio o fallido en función de si se ha sobrepasado el límite de entradas.
- **Contador atómico** (base de datos): Guarda un registro de cuantas entradas se han comprado hasta el momento.

El flujo de pasos en esta arquitectura se puede ver más claramente en la siguiente imagen.

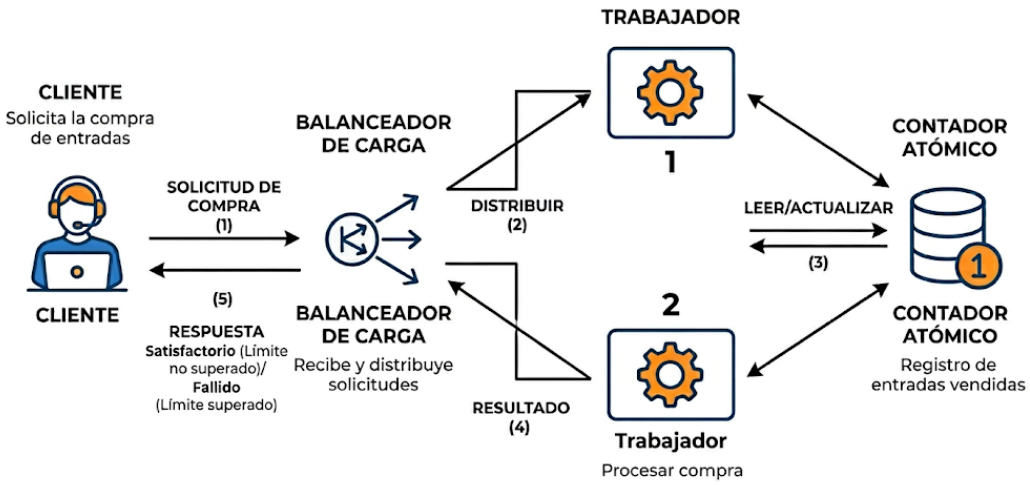

Ilustración 1 - Arquitectura para Comunicación Directa y Entradas no Numeradas

Cabe destacar que en esta arquitectura se usa un **Server Side Load Balancing**, debido a que su alternativa, Client Side Load Balancing es mucho menos escalable. Así mismo, usar un balanceador de carga aporta transparencia al sistema distribuido. Esta arquitectura sigue un enfoque Thin Client - Fat Server.

Implementación

Para implementar la arquitectura propuesta hemos hecho 3 scripts en Python: `client.py`, `loadbalancer.py` y `worker.py`.

Se ha escogido **`Pyro5`** como middleware de comunicación indirecta, debido a que es más potente que XML RPC. Además, Pyro5 usa por defecto un servicio de nombres, de modo que el cliente se conectará a un dominio. Esta característica permite desacoplamiento de localización.

Para tener el contador atómico se ha usado **`REDIS`**, una base de datos en memoria principal que destaca por su rapidez.

Algunas características para destacar de nuestra implementación son:

- El balanceador de carga aprovecha la capacidad **multihilo** de Pyro5.
- El balanceador de carga mantiene **conexiones "vivas"** con los trabajadores para que no tenga que reconectarse a ellos cada vez que envíe peticiones.
- Los trabajadores pueden **registrarse y eliminarse** en la lista dinámica del balanceador de carga en tiempo de ejecución.

Resultados

Estas pruebas se han realizado partiendo del archivo "`benchmark_unnumbered_60000.txt`" con 20.000 peticiones. Se muestran los tiempos de ejecución.

Nº de Prueba	Nº de Clientes	Nº de Trabajadores	Tiempo (s)
1	1	2	79
2	2	2	76 76
3	3	2	64 64 64
4	4	2	66 66 66 66
5	1	3	73
6	2	3	71 71
7	3	3	70 70 70
8	2	4	70 70

A pesar de las diferentes configuraciones, no hay demasiada variabilidad en los tiempos de ejecución. Se observa que, con el mismo número de trabajadores, el aumento de clientes es inversamente proporcional al tiempo de procesado del total de peticiones. Es decir, al sistema distribuido escala correctamente con la carga de trabajo.

Entradas Numeradas

Arquitectura

La arquitectura usada para la venta de entradas numeradas es la misma que para las entradas no numeradas. Sin embargo, hay un ligero cambio en cómo se interactúa con la base de datos. Anteriormente con un contador atómico era suficiente debido a que solo era necesario conocer el numero de entradas vendidas. Ahora también hay que conocer a quién se le ha vendido la entrada.

Implementación

Para implementar el registro de compra de entradas hemos decidido usar los hashes de `REDIS`. Estos son estructuras parecidas a los diccionarios, es decir, contienen un conjunto de claves con sus valores asociados. En nuestro contexto, la clave será el identificador de asiento, y el valor el identificador del cliente que ha comprado dicho asiento. Si un asiento no esta en el conjunto significa que aun no ha sido comprado.

Resultados

Estas pruebas se han realizado partiendo del archivo "`benchmark_numbered_20000.txt`" con 26.000 peticiones. Se muestran los tiempos de ejecución.

Nº de Prueba	Nº de Clientes	Nº de Trabajadores	Tiempo (s)
1	1	2	89
2	2	2	91 91
3	3	2	79 79 79
4	4	2	83 83 83 83
5	1	3	87
6	2	3	90 90
7	3	3	90 90 90
8	2	4	90 90

Se observa que en el caso de las entradas numeradas no escala tan bien como antes. Aun así, los tiempos son bastante homogéneos, es decir, no empeoran notablemente en función de la carga de trabajo.

Como requisito adicional, se pedía hacer una prueba de contención. Hemos hecho otro archivo te texto llamado `benchmark_hotspot_60000.txt` de 60.000 peticiones, que centra el 80% de ellas en el 5% de los asientos. Un solo cliente con dos trabajadores ha tardado 215s en procesar todas las peticiones y de las 60.000 se han comprado satisfactoriamente 9893, el resto de las peticiones apuntaban a asientos ya comprados.

Comunicación Indirecta

Entradas No Numeradas

Arquitectura

En este tipo de comunicación se requería un middleware orientado a mensajes para desacoplar el envío y la recepción de peticiones. Hemos usado la arquitectura basada en el patrón Publisher-Subscriber. La arquitectura consta de los siguientes actores:

- **Cliente:** Solicita la compra de entradas publicando mensajes de forma masiva y se queda a la escucha en una cola temporal exclusiva para recibir los resultados.
- **Broker de Mensajería:** Se ha usado RabbitMQ. Este actúa como intermediario y balanceador. Recibe las solicitudes en una cola principal y las distribuye entre los trabajadores suscritos. Al recibir los resultados de los trabajadores, los enruta de vuelta a la cola temporal del cliente.
- **Workers:** Consumen peticiones de la cola principal e incrementan el contador atómico. El resultado de la compra es satisfactorio o fallido en función de si se ha sobrepasado el límite de entradas, tras lo cual envían la respuesta al broker y confirman el procesamiento con un ACK.
- **Contador atómico:** Guarda un registro de cuantas entradas se han comprado hasta el momento.

El flujo de pasos en esta arquitectura se puede ver más claramente en la siguiente imagen.

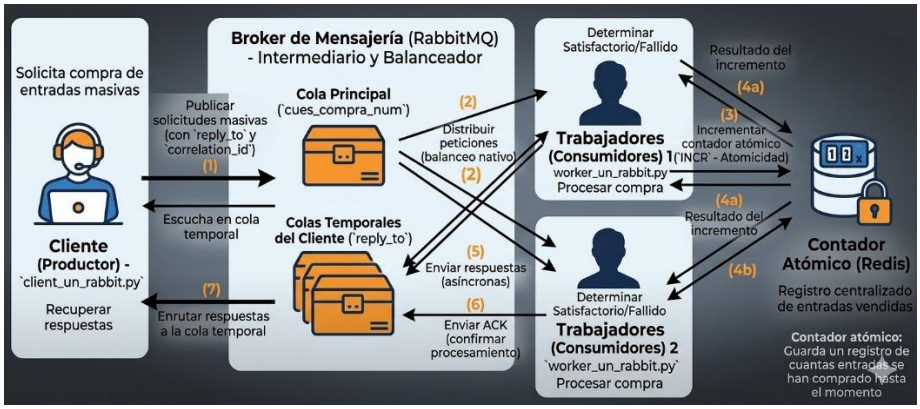

Ilustración 2 - Arquitectura para Comunicación Inirecta y Entradas no Numeradas

Cabe destacar que en esta arquitectura el balanceo de carga se realiza de forma implícita y nativa por el propio broker. Debido a que el cliente no conoce la ubicación ni la cantidad de trabajadores, la alternativa indirecta aporta un alto grado de

desacoplamiento al sistema distribuido, favoreciendo la tolerancia a fallos y eliminando los cuellos de botella directos en la red del cliente.

Implementación

Para implementar la arquitectura propuesta hemos hecho 3 scripts en Python: `client_un_rabbit.py`, `worker_un_rabbit.py` y un script de inicialización `reset_db.py`.

Se ha escogido `RabbitMQ` como middleware de comunicación indirecta, debido a su robustez y rapidez. Para tener el contador atómico, al igual que en el modelo de comunicación indirecta se ha usado `REDIS`.

Algunas características para destacar de nuestra implementación son:

- El sistema implementa un control de flujo para asignar mensajes en bloques controlados, evitando la saturación de memoria en caso de que haya workers con menor capacidad de procesamiento que otros
- Se utiliza un patrón RPC asíncrono para el retorno de estados, enviando en cada petición una cola de respuesta "reply_to" y un identificador único "correlation_id".
- Los trabajadores pueden inicializarse y destruirse dinámicamente en tiempo de ejecución. El sistema mantiene la corrección y escala su throughput de forma transparente, reasignando mensajes no confirmados automáticamente si un trabajador sufre una caída.

Resultados

Estas pruebas se han realizado partiendo del archivo "`benchmark_unnumbered_20000.txt`" con 20.000 peticiones. Se muestran los tiempos de ejecución.

Nº de Prueba	Nº de Clientes	Nº de Trabajadores	Tiempo (s)
1	1	2	37
2	2	2	47 46
3	3	2	57 57 57
4	4	2	70 70 70 70
5	1	3	37
6	2	3	47 47
7	3	3	57 57 56
8	2	4	47 46

En primer lugar, al mantener fijo el número de trabajadores y aumentar el número de clientes concurrentes, el tiempo de procesamiento experimenta un incremento gradual. El hecho de que no aumente el tiempo indica que el sistema se puede escalar.

Por otro lado, el aumentar el número de workers no reduce los tiempos de ejecución. Esto se debe a que el cuello de botella se sitúa en el cliente y su capacidad de enviar mensajes y recibir ACKs. Se puede llegar a esta conclusión debido a que en pruebas realizadas en otro entorno con 2 máquinas diferentes y mismas configuraciones los tiempos ejecución eran menores.

Entradas Numeradas

Arquitectura

En este tipo de comunicación, además de requerir un middleware orientado a mensajes para el desacoplamiento, se introduce la tarea de gestionar entradas con numeración. Se mantiene la arquitectura basada en el patrón Publisher. La diferencia principal es que ahora el cliente envía el número de asiento que quiere adquirir y que HSETNX remplaza al contador atómico global. Este guarda un registro individual de cada asiento vendido hasta el momento, mapeando el identificador del asiento con el del cliente que lo ha adquirido.

Al igual que en el modelo no numerado, el balanceo de carga se realiza de forma implícita por el bróker.

Implementación

Para implementar la arquitectura propuesta hemos hecho 3 scripts en Python: `client_num_rabbit.py`, `worker_num_rabbit.py` y un script de inicialización `reset_db_num.py`.

Se ha escogido RabbitMQ como middleware de comunicación indirecta. A diferencia del contador atómico del modelo de entradas no numeradas, se ha usado REDIS gestionando una estructura de datos Hash que mapea cada asiento con su comprador. Algunas características para destacar de nuestra implementación son:

- Se utiliza el comando atómico HSETNX de REDIS para evitar condiciones de carrera
- Se utiliza un patrón RPC asíncrono para el retorno de estados, enviando en cada petición una cola de respuesta "reply_to" y un identificador único "correlation_id".
- Los trabajadores pueden inicializarse y destruirse dinámicamente en tiempo de ejecución. El sistema mantiene la corrección y escala su throughput de forma transparente, reasignando mensajes no confirmados automáticamente si un trabajador sufre una caída.

Resultados

Estas pruebas se han realizado partiendo del archivo "`benchmark_numbered_60000.txt`" con 26.000 peticiones. Se muestran los tiempos de ejecución.

Nº de Prueba	Nº de Clientes	Nº de Trabajadores	Tiempo (s)
1	1	2	50
2	2	2	61 61
3	3	2	67 69 69
4	4	2	92 91 92
5	1	3	49
6	2	3	61 62
7	3	3	74 74 74
8	2	4	61 61

Se puede observar que igual que en las no numeradas al aumentar el número de clientes el sistema absorbe la carga el aumento de concurrencia de forma estable.

Por otra parte, al aumentar el número de trabajadores se puede observar que sucede el mismo suceso que en las pruebas anteriores. El número de workers no afecta al tiempo de ejecución. Esto también se debe a la velocidad de envío de peticiones y del cliente.

En esta arquitectura también se ha hecho una prueba de contención con 1 cliente y 2 workers. Ha tardado 116s en procesar todas las peticiones y de las 60.000 se han comprado satisfactoriamente 9893, el resto de las peticiones apuntaban a asientos ya comprados.